

“PROJECT REPORT”

AQI PREDICTOR

An End-to-End, Serverless Machine Learning System for 3-Day Air
Quality Forecasting Across 19 Pakistani Cities

Submitted by: Muhammad Bin Junaid

Company: 10Pearls

Program: Bachelor of Science in Computer Science

Field: Data Science Intern

Date: 6 September 2026

1. Project Overview

Air pollution is a persistent and worsening problem across Pakistan's major cities, with air quality index (AQI) readings frequently reaching unhealthy or hazardous levels. This project, AQI Predictor, is an end-to-end, fully serverless machine learning system that forecasts the Air Quality Index up to three days in advance for 19 major Pakistani cities: Karachi, Lahore, Islamabad, Rawalpindi, Faisalabad, Multan, Peshawar, Quetta, Sialkot, Gujranwala, Hyderabad, Sukkur, Bahawalpur, Sargodha, Sheikhupura, Larkana, Gujrat, Mardan, and Kasur.

The system was built around four core components required by the project specification: (1) a feature pipeline that continuously ingests and engineers data, (2) a training pipeline that trains and evaluates multiple forecasting models, (3) a fully automated CI/CD pipeline that keeps both of the above running on a schedule without manual intervention, and (4) an interactive web dashboard that surfaces real-time and forecasted AQI values to end users, complete with model-explainability and hazard alerting.

The guiding design principle throughout was to use a 100% serverless stack: no self-managed servers or databases, relying instead on managed free-tier services (Hopsworks for the feature store and model registry, GitHub Actions for orchestration, and Streamlit Cloud for hosting the dashboard).

1.1 System Architecture

At a high level, data flows through the system as follows:

- Open-Meteo API → Feature Pipeline (Python) → Hopsworks Feature Store
- Hopsworks Feature Store → Training Pipeline (Python) → Hopsworks Model Registry
- Hopsworks Feature Store + Model Registry → Streamlit Dashboard → End User
- GitHub Actions → triggers the Feature Pipeline hourly and the Training Pipeline daily, keeping the whole system self-updating

This architecture cleanly separates concerns: feature engineering, model training, orchestration, and presentation are independent scripts that only communicate through the Hopsworks Feature Store and Model Registry, which act as the system's single source of truth.

2. Data Collection and Feature Engineering

2.1 Choice of Data Source

The project specification suggested AQICN or OpenWeather as example data sources but explicitly encouraged exploring alternatives. In practice, the data source went through three iterations before settling on a final choice:

- AQICN was tried first, but its free tier had limited station coverage across the required Pakistani cities, meaning many cities either had no station or unreliable readings.
- OpenWeather was tried next, but its air-pollution endpoints required account activation delays and did not offer sufficient free historical data for backfilling two years of training data.
- Open-Meteo was ultimately selected as the final data source. It provided free, immediate access to both current and historical weather and air-quality data for arbitrary coordinates, with no activation delay and no practical rate-limit issues for this project's scale.

This pivot is treated as a genuine engineering decision rather than a setback: each earlier option solved part of the problem but failed a specific hard requirement (either city coverage or historical depth), and Open-Meteo was chosen because it was the only option satisfying both.

2.2 Feature Pipeline

The feature pipeline (`feature_pipeline.py`) is the script responsible for turning raw API data into model-ready features. It supports two run modes:

- `--mode live`: fetches the current weather and pollutant readings for all 19 cities and appends them to the feature store. This mode is triggered hourly by GitHub Actions.
- `--mode backfill --days N`: loops over the past N days for every city, re-fetching historical data so the feature store contains a continuous historical record suitable for training.

For each city and timestamp, the pipeline computes:

- Raw pollutant readings (e.g. PM2.5, PM10, and related pollutant concentrations) and raw weather variables from Open-Meteo.
- Time-based features: hour of day, day of week, and month, allowing the models to learn diurnal and seasonal pollution patterns.
- A derived feature, `aqi_change_rate`, capturing how quickly AQI is rising or falling, which is informative for short-horizon forecasting.

All engineered rows are written to a Hopsworks Feature Store feature group named `aqi_features` (schema version 4 at the time of writing), which is the shared, versioned interface that both the training pipeline and the dashboard read from.

2.3 Historical Data Backfill

To build a training dataset large enough for reliable multi-day forecasting, the feature pipeline was run in backfill mode across the full set of 19 cities for a period of 730 days (two years). This produced a final training dataset of approximately 336,984 rows, giving the models enough historical depth to learn seasonal AQI patterns rather than only short-term noise.

Note: an earlier attempt used only a 90-day backfill window. This proved insufficient (see Section 6.3) and was the direct motivation for the full two-year backfill described above.

3. Exploratory Data Analysis

Before modelling, an EDA script (`notebooks/eda.py`) was used to understand distributions, trends, and correlations in the backfilled dataset. Key findings included:

- The overall mean AQI across all 19 cities and the full two-year window was approximately 126, placing the average reading in the “unhealthy for sensitive groups” range on standard AQI scales.
- Not a single reading in the dataset fell into the “Good” AQI category, underscoring how persistently poor air quality is across the sampled cities.
- AQI exhibits a clear midday peak pattern across most cities, consistent with daytime traffic and industrial activity.
- City-level rankings showed Kasur and Faisalabad as consistently the most polluted cities in the dataset, while Karachi and Hyderabad were consistently the least polluted of the 19.
- PM2.5 and PM10 emerged as the strongest correlates of overall AQI, ahead of other pollutant and weather variables, which directly informed feature importance expectations later validated by SHAP (Section 7).

These findings both validated the choice of engineered features (time-based and rate-of-change features) and gave an independent sanity check for the explainability results produced later in the project.

4. Model Training and Evaluation

4.1 Training Pipeline

- Fetches the historical (features, target) pairs from the Hopsworks Feature Store.
- Trains and evaluates multiple model types for three forecast horizons: 24 hours, 48 hours, and 72 hours ahead, giving a genuine 3-day forecast rather than a single-point prediction.
- Stores each trained model in the Hopsworks Model Registry, versioned per horizon, so the dashboard can always load the latest model for each forecast window.

A chronological (time-ordered) train/test split was used rather than a random shuffle. This is a deliberate choice for time-series data: shuffling would let the model “see the future” during training and produce misleadingly good validation scores that would not hold up on genuinely unseen future data.

4.2 Models Compared

The project specification called for a range of forecasting approaches from statistical modelling to deep learning. Three model families were implemented and compared:

Model	Category	Notes
Ridge Regression	Statistical / Linear	Fast baseline; coefficients double as an explainability method (Section 7).
Random Forest	Classical machine learning	scikit-learn ensemble; captures non-linear feature interactions.
TensorFlow dense network	Deep learning	Small fully connected network; the most flexible but most data-hungry model.

Each model was trained and evaluated independently for each of the three forecast horizons (24h / 48h / 72h), using RMSE (Root Mean Squared Error), MAE (Mean Absolute Error), and R^2 (coefficient of determination) as evaluation metrics — exactly as required by the project specification.

4.3 Results and the 90-Day vs. Two-Year Finding

The single most instructive result from the training phase came from comparing two different backfill windows:

Training Data Window	Result
90 days of history	Negative R^2 on the test set, i.e., the model performed worse than simply predicting the historical average AQI.
Full 2-year (730-day) backfill	R^2 of approximately 0.633 / 0.376 / 0.324 across the 24h / 48h / 72h horizons respectively.

This result is treated as a genuine data-science finding rather than merely a bug fix: with only 90 days of data, the models had not seen enough seasonal variation to generalise, and effectively memorised noise. Extending the backfill to two years gave the models enough historical range to learn real seasonal and city-level patterns, which is reflected in the large jump from negative to positive, and progressively more valid R^2 scores. As expected, accuracy degrades gracefully as the forecast horizon increases (24h is the most accurate, 72h the least), which is consistent with the inherent difficulty of longer-range forecasting.

Overall, the TensorFlow model represents the deep-learning end of the required model spectrum, Random Forest the classical machine-learning middle ground, and Ridge Regression the statistical baseline satisfying the specification's requirement for “a variety of forecasting models, from statistical modelling to deep learning.”

5. Automated CI/CD Pipeline

To satisfy the requirement for a scalable, automated pipeline, both the feature pipeline and the training pipeline are orchestrated entirely through GitHub Actions, with no manual steps required after initial setup:

Workflow	Schedule	Purpose
Feature Pipeline	Hourly	Fetches the latest live weather/pollutant data for all 19 cities and appends engineered features to the Hopsworks Feature Store.
Training Pipeline	Daily	Retrains all three model types across all three horizons on the latest feature-store data and pushes updated models to the Hopsworks Model Registry.

Both workflows were manually triggered and verified to complete successfully (green checkmark) in the GitHub Actions tab before being considered production-ready, satisfying the specification's requirement that the pipeline actually run not merely be configured to run.

Getting both workflows to run reliably required diagnosing and fixing several real environment issues, documented in full in Section 6.

6. Web Application / Dashboard

The dashboard (dashboard.py) is built with Streamlit and deployed on Streamlit Cloud, satisfying the specification's requirement for a Streamlit/Gradio/Flask/FastAPI web app. On load, it:

- Connects to Hopsworks and loads the latest engineered features and the latest registered model for each forecast horizon.
- Computes and displays real-time AQI alongside 24h / 48h / 72h forecasts for the selected city.
- Displays a city leaderboard ranking all 19 cities by current or forecasted AQI.
- Shows a hazardous-AQI alert banner whenever a forecast reaches $AQI \geq 151$ (the “Unhealthy” threshold), directly satisfying the specification's alerting requirement.
- Includes an “AQI Copilot” feature and a “Why this forecast” SHAP explainability panel (detailed in Section 7).

Configuration (the Hopsworks API key, project name, and the list of 19 tracked cities) is supplied via Streamlit Cloud's Secrets manager rather than being hard-coded, so credentials are never committed to the repository.

7. Explainability (SHAP)

The specification required using SHAP or LIME for feature-importance explanations. This was implemented using real SHAP values computed from the actual trained model for each model type, rather than a static or placeholder explanation:

Model	SHAP Method	Why
Random Forest	TreeExplainer	Exact and fast for tree-based ensembles.
TensorFlow network	KernelExplainer	Model-agnostic; slower but valid for neural networks.
Ridge Regression	Model coefficients	Linear coefficients are themselves an exact, interpretable feature-importance measure.

The dashboard's “Why this forecast” panel renders these SHAP values as a horizontal bar chart, ranking features such as PM2.5, current AQI, and hour of day by their numeric contribution to the specific prediction being shown. This complements, rather than duplicates, the RMSE/MAE/R² evaluation metrics from Section 4: the evaluation metrics answer “how much can I trust this model overall?” while the SHAP values answer “why did it predict this specific number, right now, for this specific city?”

8. Challenges and Solutions

Building and automating this pipeline surfaced a number of real, non-trivial engineering problems. Each is documented below as a case study, since the debugging process itself is a substantial part of the project's learning outcomes.

8.1 Missing pyarrow Dependency in GitHub Actions

What broke: Both the feature-pipeline and training-pipeline GitHub Actions workflows failed during dependency installation / feature-store interaction.

Why: The Hopsworks client library (hsfs) depends on pyarrow for efficient data transfer, but pyarrow was not explicitly listed in either workflow's pip install command, so it was missing from the GitHub Actions runner environment even though it may have been present locally.

How it was diagnosed: The GitHub Actions run logs were inspected directly, showing the missing-dependency error at the point where the workflow tried to insert or read data from the feature store.

Fix: pyarrow was added explicitly to the pip install line in both training_pipeline.yml and feature_pipeline.yml, after which both workflows were rerun and confirmed to complete successfully.

8.2 Hopsworks API Key Missing the SERVING Scope

What broke: The dashboard failed to serve predictions in production even though the pipelines themselves worked.

Why: The Hopsworks API key originally generated for the deployed app did not include the SERVING scope, which is required to fetch models/features for online inference rather than just batch access.

How it was diagnosed: The error pointed to a permissions/scope issue when the dashboard attempted to load the model for serving, which was traced back to the specific scopes selected when the API key was created.

Fix: A new Hopsworks API key was generated with “select all” scopes (ensuring SERVING was included), and the corresponding secret was updated in the Streamlit Cloud Secrets editor.

8.3 Windows / TensorFlow Python-Version Conflicts

What broke: TensorFlow installation and/or execution issues on the local Windows development environment.

Why: TensorFlow releases only support specific Python version ranges, and the local Python installation and virtual environment did not initially align with a supported version, causing installation or import failures.

How it was diagnosed: Installation errors and import failures were traced to Python/TensorFlow version incompatibility by checking the installed Python version against TensorFlow's supported version matrix.

Fix: The virtual environment was rebuilt against a supported Python version before reinstalling the project's dependencies (pandas, numpy, scikit-learn, tensorflow, hopsworks, joblib, pyarrow, python-dotenv, confluent-kafka).

8.4 Hopsworks Query Service Failure

What broke: A query against the Hopsworks Feature Store failed at the query-service layer rather than returning data.

Why: This surfaced as an infrastructure-level failure on Hopsworks' managed query service when reading feature data back out for training.

How it was diagnosed: The stack trace was inspected to confirm the failure originated in Hopsworks' query-service layer rather than in the project's own code.

Fix: The query was retried after adjusting the read path/connection handling, after which the feature store read succeeded normally.

8.5 Model Version-Pinning Bug

What broke: The dashboard occasionally loaded an unexpected or outdated model version instead of the latest trained model for a given horizon.

Why: The Hopsworks Model Registry keeps every trained version, and the loading code needed to explicitly resolve the latest version per horizon rather than assuming a default; without an explicit version pin/lookup this could silently serve a stale model.

How it was diagnosed: Predictions were compared against the metrics logged for the most recently trained model and found to mismatch, which was traced back to how the model version was being resolved at load time.

Fix: The model-loading code was updated to explicitly fetch the latest version per horizon from the Model Registry before serving predictions.

8.6 Git / Version Control Setup

What broke: Initial attempts to push the project to GitHub failed because the project folder had not yet been initialised as a Git repository, and later pushes were occasionally rejected.

Fix: The repository was initialised with git init, and rejected pushes were resolved with the standard git pull origin main followed by git push workflow before committing dependency and workflow fixes.

9. Requirements Compliance Summary

The table below maps every requirement in the original project specification to what was actually built and verified.

9.1 Feature Pipeline

Requirement	Status	Evidence
Fetch raw weather + pollutant data from an external API	Done	Open-Meteo API integration
Time-based features (hour, day, month)	Done	Computed in feature_pipeline.py
Derived features (e.g., AQI change rate)	Done	aqi_change_rate feature
Store features in a Feature Store	Done	Hopsworks feature group aqi_features v4

9.2 Historical Backfill

Requirement	Status	Evidence
Run feature script for a range of past dates	Done	--mode backfill --days 730
Comprehensive training dataset	Done	336,984 rows across 19 cities, 2 years

9.3 Training Pipeline

Requirement	Status	Evidence
Fetch historical features/targets from the Feature Store	Done	train_model.py
Train and evaluate the best possible model	Done	Compared 3 model families across 3 horizons
Scikit-learn models (Random Forest, Ridge)	Done	Both implemented and evaluated
TensorFlow / PyTorch model	Done	TensorFlow dense network
RMSE, MAE, R ² evaluation	Done	Computed for every model, every horizon
Store trained model in a Model Registry	Done	Hopsworks Model Registry, 3 horizons

9.4 Automated CI/CD Pipeline

Requirement	Status	Evidence
Feature script runs hourly	Done	GitHub Actions workflow, verified green
Training script runs daily	Done	GitHub Actions workflow, verified green after pyarrow fix
CI/CD tool used	Done	GitHub Actions

9.5 Web App Dashboard

Requirement	Status	Evidence
Loads model + features from the Feature Store	Done	dashboard.py
Computes real-time predictions	Done	Live inference on dashboard load
Built with Streamlit/Gradio/Flask/FastAPI	Done	Streamlit, deployed on Streamlit Cloud

9.6 Guidelines

Requirement	Status	Evidence
EDA to identify trends	Done	notebooks/eda.py — Section 3
Variety of models: statistical → deep learning	Done	Ridge → Random Forest → TensorFlow
SHAP or LIME for feature importance	Done	Real SHAP values — Section 7
Hazardous AQI alerts	Done	Banner triggers at AQI ≥ 151

9.7 Final Submission Items

Deliverable	Status
1. End-to-end AQI prediction system	Done
2. A scalable, automated pipeline	Done
3. Interactive dashboard (real-time + forecasted)	Done
4. Detailed project report	Done (this document)

10. Conclusion and Future Work

10.1 Conclusion

AQI Predictor successfully delivers on every component of the project specification: a serverless feature pipeline backed by a real, external data source; a two-year backfilled training dataset; three model families spanning statistical, classical machine-learning, and deep-learning approaches, evaluated rigorously with RMSE/MAE/ R^2 across three forecast horizons; a fully automated hourly/daily CI/CD pipeline verified to run successfully on GitHub Actions; and an interactive, explainable, alert-enabled Streamlit dashboard deployed publicly on Streamlit Cloud. Beyond meeting the checklist, the project surfaced a genuine data-science insight — that a 90-day training window was insufficient and produced a negative R^2 , whereas a full two-year backfill was necessary to achieve meaningful forecasting accuracy along with several real infrastructure and dependency issues that were diagnosed and resolved end-to-end.

10.2 Limitations

- Forecast accuracy decreases at longer horizons (72h R^2 of ~ 0.324 versus 24h R^2 of ~ 0.633), which is expected but leaves room for improvement.
- The KernelExplainer used for the TensorFlow model's SHAP values is an approximation rather than an exact method, unlike the TreeExplainer used for Random Forest.
- Data currently spans two years for 19 cities; a longer history or additional cities would likely further improve seasonal generalisation.

10.3 Future Work

- Incorporate additional historical seasons of data to further improve long-horizon (48h/72h) forecasting accuracy.
- Integrate additional pollutant data sources to cross-validate Open-Meteo readings and fill any coverage gaps.
- Extend the dashboard with a mobile-friendly interface and push notifications for hazardous AQI alerts.
- Experiment with sequence models (e.g. LSTM/Temporal Convolutional Networks) that explicitly model time-series structure, as a deep-learning alternative to the current dense network.